TD 3

Programmation Concurrente en Java

Programmation Java Avancée : POO et JVM

Rapport de travaux pratiques

Module	STI 4A-FISE 2026/2027
Enseignante	Dr. Nadia Badri Fettah
Durée	1h20
Code source	`https://github.com/JamaiAli/td3java`

19 octobre 2026

Table des matières

Objectifs du TD **2**

1 Téléchargements : séquentiel ou parallèle ? **2**
1.1 Question 1.1 — La tâche (`Telechargement.java`) . . . 2
1.2 Question 1.2 — Version séquentielle . . . 2
1.3 Question 1.3 — Version parallèle (`Navigateur.java`) . . . 2
1.4 Question 1.4 — Sans `join()` . . . 3
1.5 Question 1.5 — Tableau comparatif . . . 3

2 Cycle de vie : sleep, join, getState, daemon **3**
2.1 Question 2.1 — Afficher les états (`Etats.java`) . . . 3
2.2 Question 2.2 — Le rôle de `join()` . . . 4
2.3 Question 2.3 — Redémarrer un thread terminé . . . 4
2.4 Question 2.4 — Le thread daemon (`Horloge.java`) . . . 4

3 Une ArrayList partagée qui perd des éléments **4**
3.1 Question 3.1 — Le bug . . . 4
3.2 Question 3.2 — Comprendre . . . 5
3.3 Question 3.3 — Correction n°1 : un bloc `synchronized` . . . 5
3.4 Question 3.4 — Correction n°2 : une liste synchronisée . . . 5
3.5 Question 3.5 — Réflexion . . . 5

4 Bloc `synchronized` et verrous distincts **6**
4.1 Question 4.1 — Deux ressources, un seul verrou . . . 6
4.2 Question 4.2 — Un verrou par ressource . . . 6
4.3 Question 4.3 — Réflexion . . . 6

5 `volatile` et `AtomicInteger` **6**
5.1 Question 5.1 — Arrêter proprement un thread (`Travailleur.java`) . . . 6
5.2 Question 5.2 — La correction . . . 6
5.3 Question 5.3 — `volatile` ne suffit pas pour `++` . . . 7
5.4 Question 5.4 — `AtomicInteger` . . . 7
5.5 Question 5.5 — Tableau récapitulatif . . . 7

6 ExecutorService, Callable et Future **7**
6.1 Question 6.1 — Un pool de 3 threads (`PoolSimple.java`) . . . 7
6.2 Question 6.2 — Callable et Future (`SommeFuture.java`) . . . 8
6.3 Question 6.3 — invokeAll (`SommeParallele.java`) . . . 8
6.4 Question 6.4 — ScheduledExecutorService (`Bip.java`) . . . 8

7 Le départ de la course **9**
7.1 Étape 1 — `LigneDeDepart` (TODO 1 à 4) . . . 9
7.2 Étapes 3 et 4 — `Coureur` et `Course` . . . 9
7.3 Étape 2 — Question sur `Classement` . . . 10
7.4 Étape 5 — Expériences . . . 10
7.5 Étape 6 — Comprendre les choix . . . 10
7.6 La version moderne — `CountDownLatch` (bonus) . . . 10

8 FAQ — Erreurs fréquentes rencontrées **11**

Conclusion **11**

Objectifs du TD

- Créer et démarrer des threads avec `Runnable` et avec `extends Thread`.
- Observer le cycle de vie d'un thread (`getState`, `sleep`, `join`, `daemon`).
- Mettre en évidence une *race condition* et la corriger avec `synchronized`.
- Utiliser `volatile`, `AtomicInteger`, `ExecutorService`, `Callable` et `Future`.
- Synchroniser le départ de plusieurs threads avec `wait()` et `notifyAll()`.

> *Chaque capture d'écran provient d'une exécution réelle du programme dans IntelliJ IDEA ; les valeurs citées dans le texte (durées, tailles, résultats) correspondent à ces exécutions. Comme l'ordre d'exécution des threads varie, les valeurs exactes peuvent différer d'une exécution à l'autre.*

1 Téléchargements : séquentiel ou parallèle ?

On simule 3 téléchargements : `photo.jpg` (2 s), `video.mp4` (3 s) et `cours.pdf` (1 s). Un « téléchargement » est une simple attente avec `sleep()`.

1.1 Question 1.1 — La tâche (`Telechargement.java`)

```java
public class Telechargement implements Runnable {
    private String fichier;
    private int secondes;

    public Telechargement(String fichier, int secondes) {
        this.fichier = fichier;
        this.secondes = secondes;
    }

    public void run() {
        String qui = Thread.currentThread().getName();
        System.out.println(qui + " commence " + fichier);
        for (int i = 1; i <= secondes; i++) {
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) { }
            System.out.println("   " + fichier + " : " + (i * 100 / secondes) + " %");
        }
        System.out.println(qui + " a terminé " + fichier);
    }
}
```

1.2 Question 1.2 — Version séquentielle

On appelle `run()` directement : tout est exécuté par le thread `main`, les fichiers se téléchargent l'un après l'autre. **Durée ≈ 6000 ms (2 + 3 + 1).**

```java
d1.run();
d2.run();
d3.run();
```

1.3 Question 1.3 — Version parallèle (`Navigateur.java`)

On crée un thread par téléchargement, on les démarre avec `start()`, puis on attend leur fin avec `join()`.

```java
Thread t1 = new Thread(d1, "DL-1");
Thread t2 = new Thread(d2, "DL-2");
Thread t3 = new Thread(d3, "DL-3");

t1.start(); t2.start(); t3.start();
t1.join();  t2.join();  t3.join();
```

À l'exécution (voir capture), les pourcentages des trois fichiers sont mélangés et la durée totale est $\approx$ **3000 ms** (le plus long des trois), contre $\approx$ 6000 ms en séquentiel.

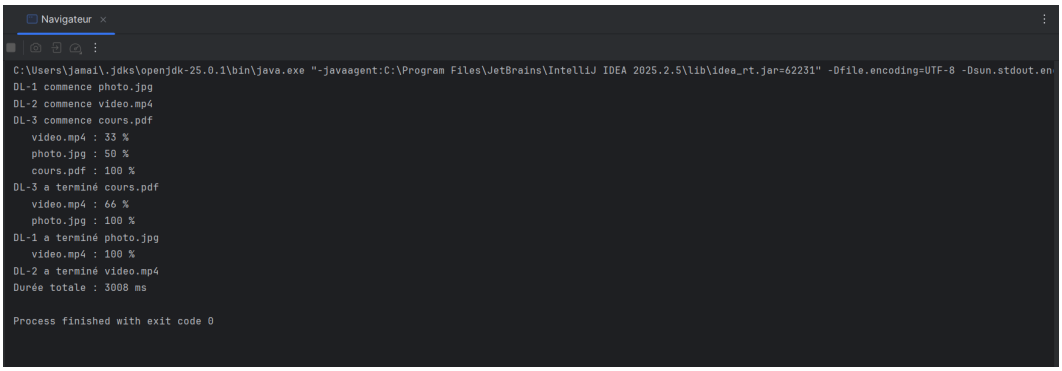


FIG. 1 : Exécution parallèle de `Navigateur` — durée $\approx$ 3000 ms

1.4 Question 1.4 — Sans `join()`

Si on met les trois `join()` en commentaire, la durée affichée tombe à $\approx$ 0 ms. C'est **faux** : le `main` n'attend plus la fin des téléchargements, il calcule la durée immédiatement après avoir lancé les threads.

1.5 Question 1.5 — Tableau comparatif

Version	Durée mesurée (ms)	Thread qui exécute
Séquentielle (`run()`)	$\approx$ 6000	`main`
Parallèle (`start()`+`join()`)	$\approx$ 3008	`DL-1`, `DL-2`, `DL-3`

Pourquoi la version parallèle est-elle plus rapide, même sur un seul cœur ? Parce que pendant un `sleep()`, un thread **n'utilise pas le processeur**. Les trois attentes se déroulent en même temps au lieu de s'additionner, d'où une durée égale au plus long ($\approx$ 3 s) et non à leur somme ($\approx$ 6 s).

2 Cycle de vie : sleep, join, getState, daemon

2.1 Question 2.1 — Afficher les états (`Etats.java`)

```java
System.out.println("1. " + t.getState()); // avant start()  -> NEW
t.start();
System.out.println("2. " + t.getState()); // juste après start() -> RUNNABLE
Thread.sleep(200);
System.out.println("3. " + t.getState()); // t dort -> TIMED_WAITING
t.join();
System.out.println("4. " + t.getState()); // après fin -> TERMINATED
```

À l'exécution, les états s'enchaînent dans l'ordre attendu : `NEW` → `RUNNABLE` → `TIMED_WAITING` → `TERMINATED` (voir capture).

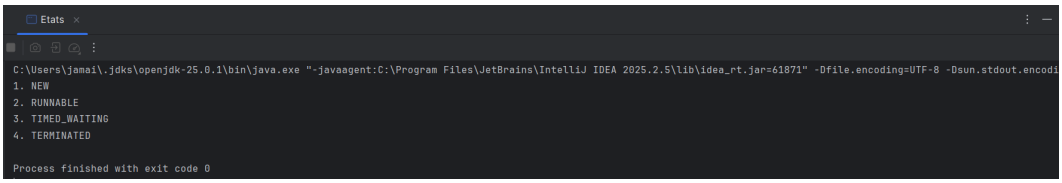


FIG. 2 : États successifs du thread (NEW, RUNNABLE, TIMED_WAITING, TERMINATED)

2.2 Question 2.2 — Le rôle de `join()`

Si on commente le `join()` juste avant la ligne 4, celle-ci peut afficher `TIMED_WAITING` (le thread dort encore) au lieu de `TERMINATED`. **`join()` bloque le thread appelant jusqu'à ce que le thread visé soit réellement terminé.**

2.3 Question 2.3 — Redémarrer un thread terminé

Ajouter `t.start();` à la fin du `main` lève `java.lang.IllegalThreadStateException` : **un thread ne peut être démarré qu'une seule fois**. Une fois `TERMINATED`, il faut créer un nouvel objet `Thread`.

2.4 Question 2.4 — Le thread daemon (`Horloge.java`)

- **Étape a** (sans `setDaemon`) : la boucle `while(true)` ne s'arrête jamais ; il faut `Ctrl + C`.
- **Étape b** (`t.setDaemon(true);` *avant* `start()`) : ∼4 lignes `Tic`, puis `Fin du main`, puis arrêt automatique. Quand il ne reste que des threads daemon, la JVM se termine.
- **Étape c** (`setDaemon(true)` *après* `start()`) : lève `IllegalThreadStateException`.

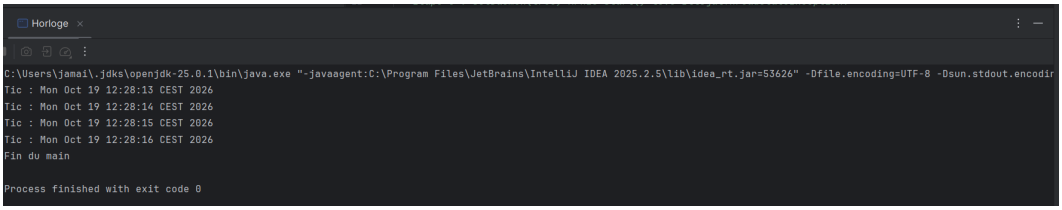


FIG. 3 : Horloge daemon : arrêt après « Fin du main »

3 Une ArrayList partagée qui perd des éléments

3.1 Question 3.1 — Le bug

Deux threads ajoutent chacun 10 000 entiers dans la même `ArrayList` non protégée (taille attendue 20000). On obtient des tailles **inférieures à 20000** et parfois une `ArrayIndexOutOfBoundsException`.

Exécution	Taille obtenue	Exception ?
1 (capture)	12706	non
2	11285	non
3	10000	oui (AIOOBE)
4	11442	non
5	10004	oui (AIOOBE)

Les valeurs varient à chaque exécution (l'exécution 1 est celle de la capture). On n'obtient jamais 20000, et une `ArrayIndexOutOfBoundsException` apparaît par moments. *Si vous obtenez toujours 20000, remplacez `10_000` par `100_000` (taille attendue 200000).*

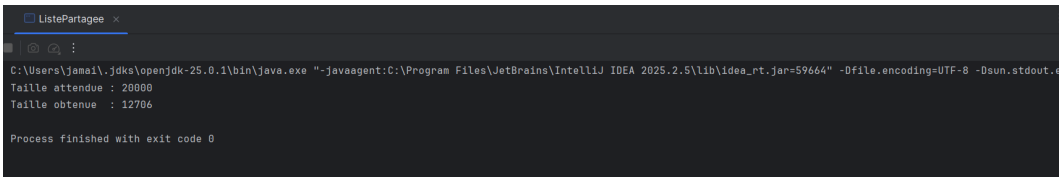


FIG. 4 : Taille < 20000 et/ou ArrayIndexOutOfBoundsException

3.2 Question 3.2 — Comprendre

`liste.add(x)` fait deux choses : (1) ranger `x` dans la case n° `size` ; (2) `size++`. Ces étapes n'étant pas atomiques, deux threads peuvent lire la **même** valeur de `size`, écrire dans la **même** case (l'un écrase l'autre → élément perdu), puis incrémenter `size` deux fois → taille incohérente et case « vide » → exception.

3.3 Question 3.3 — Correction n°1 : un bloc `synchronized`

```
synchronized (liste) {
    liste.add(i);
}
```

Résultat : `Taille obtenue : 20000` à chaque exécution, sans exception (résultat attendu). On verrouille la **ressource partagée elle-même** car le `main` est `static` (pas de `this`).

3.4 Question 3.4 — Correction n°2 : une liste synchronisée

```
import java.util.Collections;
List<Integer> liste = Collections.synchronizedList(new ArrayList<>());
```

Chaque méthode (`add`, `size`, `get`…) est déjà `synchronized`. Résultat : 20000 à chaque fois.

3.5 Question 3.5 — Réflexion

```
if (!liste.contains(42)) {
    liste.add(42);
}
```

Peut-on obtenir deux fois 42 ? Oui. Chaque appel est protégé individuellement, mais **pas l'ensemble « vérifier puis ajouter »**. Deux threads peuvent évaluer `contains(42)` à `false` avant qu'aucun n'ait ajouté. **Correction :** rendre le bloc composé atomique.

```
synchronized (liste) {
    if (!liste.contains(42)) {
        liste.add(42);
    }
}
```

4 Bloc `synchronized` et verrous distincts

4.1 Question 4.1 — Deux ressources, un seul verrou

Avec `public synchronized void ...`, le verrou est **l'objet `b` tout entier** : `ajouterLivre` et `ajouterLecteur` ne peuvent pas s'exécuter en même temps. **Durée ≈ 2000 ms** (20 appels × 100 ms en série). Les deux ressources sont indépendantes, pourtant `t2` attend `t1` inutilement.

4.2 Question 4.2 — Un verrou par ressource

```
public void ajouterLivre(String titre) {
    synchronized (livres) { livres.add(titre); }
    pause();                        // hors du bloc
}
public void ajouterLecteur(String nom) {
    synchronized (lecteurs) { lecteurs.add(nom); }
    pause();
}
```

À l'exécution, la durée tombe à `Durée : 1010 ms` ≈ **1000 ms** (voir capture), contre ≈ 2000 ms avec un verrou unique : les deux threads avancent maintenant en parallèle (verrous distincts).

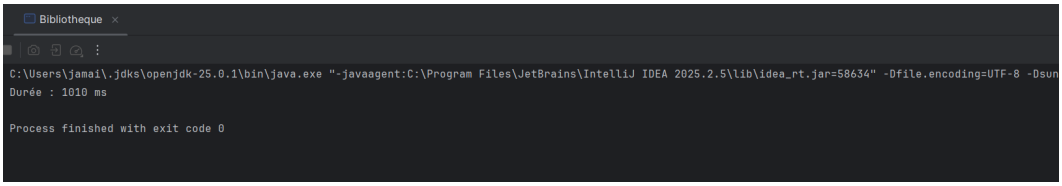


FIG. 5 : Durée ≈ 1000 ms avec un verrou par ressource

4.3 Question 4.3 — Réflexion

On laisse `pause()` hors de la section critique car garder un verrou pendant une opération longue bloquerait inutilement les autres threads ; on ne verrouille que la modification réellement partagée (l'ajout dans la liste).

5 `volatile` et `AtomicInteger`

5.1 Question 5.1 — Arrêter proprement un thread (`Travailleur.java`)

Sans `volatile`, le thread peut **ne jamais voir** que `actif` est passé à `false` (valeur mise en cache). Sur beaucoup de machines on lit alors `Le thread tourne encore !`. Le problème n'est pas garanti mais possible — ce qui le rend dangereux.

5.2 Question 5.2 — La correction

```
private volatile boolean actif = true;
```

Toute écriture de `actif` devient immédiatement visible par les autres threads : le thread s'arrête et le programme affiche `Le thread est bien arrêté` à chaque exécution (voir capture).

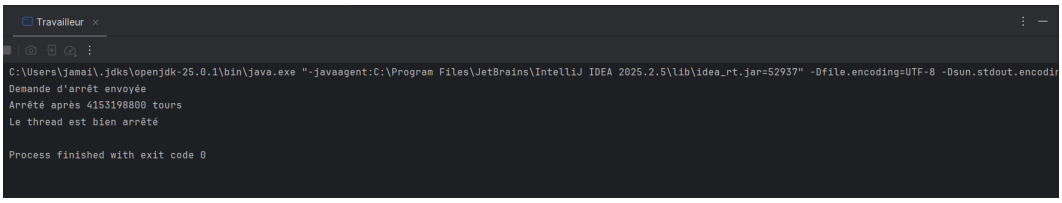


FIG. 6 : « Le thread est bien arrêté » avec `volatile`

5.3 Question 5.3 — `volatile` ne suffit pas pour `++`

Avec `private volatile int vues = 0;` et `vues++`, le résultat est **faux** (< 200000). `volatile` garantit la **visibilité** mais **pas l'atomicité** : `vues++` = *lire* $\rightarrow$ *+1* $\rightarrow$ *écrire* (3 étapes).

5.4 Question 5.4 — `AtomicInteger`

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 private AtomicInteger vues = new AtomicInteger(0);
3 public void nouvelleVue() { vues.incrementAndGet(); }
4 public int getVues() { return vues.get(); }
```

`incrementAndGet()` est **atomique** : on obtient `Vues : 200000` à chaque exécution, sans aucun `synchronized` (voir capture).

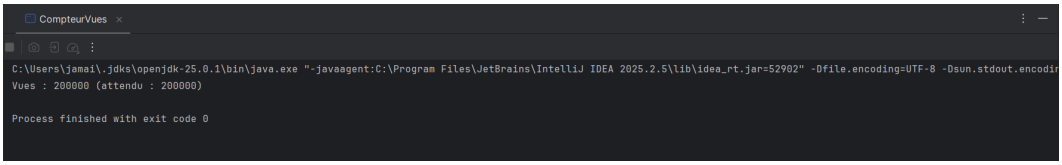


FIG. 7 : Compteur de vues avec `AtomicInteger` : 200000 (exact)

5.5 Question 5.5 — Tableau récapitulatif

Outil	Garantit la visibilité	Rend `c++` sûr
Rien	Non	Non
`volatile`	Oui	Non
`synchronized`	Oui	Oui
`AtomicInteger`	Oui	Oui

6 ExecutorService, Callable et Future

6.1 Question 6.1 — Un pool de 3 threads (`PoolSimple.java`)

Les 6 tâches sont exécutées dans un ordre variable par les threads du pool (voir capture). **Combien de threads différents ? 3** (`pool-1-thread-1`, `-2`, `-3`), réutilisés pour plusieurs tâches. **Si on oublie `pool.shutdown()` ?** Le programme **ne se termine jamais** (threads du pool non daemon en attente).

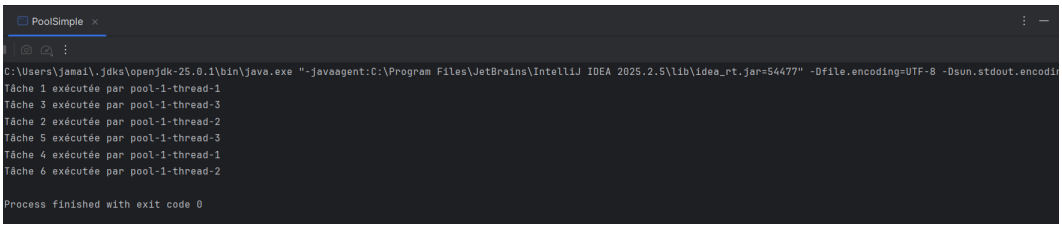


FIG. 8 : Un pool de 3 threads exécute 6 tâches

6.2 Question 6.2 — Callable et Future (`SommeFuture.java`)

```
Future<Long> f = pool.submit(somme);
System.out.println("Résultat : " + f.get());
```

On obtient `Résultat : 500000500000` (`f.get()` est bloquant : il attend la fin du calcul avant d'afficher, voir capture).

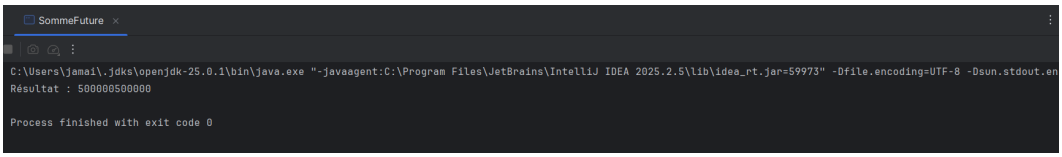


FIG. 9 : Résultat récupéré via `Future.get()` : 500000500000

6.3 Question 6.3 — invokeAll (`SommeParallele.java`)

Somme de 1 à 4 000 000 découpée en 4 morceaux.

```
List<Future<Long>> resultats = pool.invokeAll(taches);
long total = 0;
for (Future<Long> f : resultats) {
    total += f.get();
}
```

On obtient `Total : 8000002000000` (voir capture).

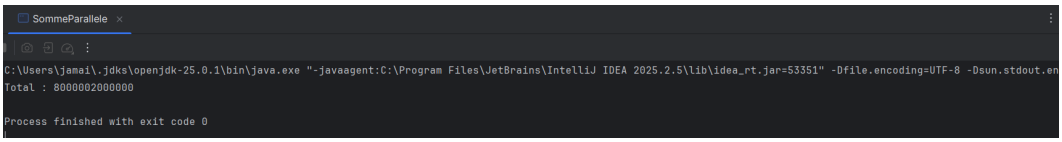


FIG. 10 : Somme parallèle via `invokeAll` : 8000002000000

6.4 Question 6.4 — ScheduledExecutorService (`Bip.java`)

```
ScheduledExecutorService pool = Executors.newScheduledThreadPool(1);
pool.scheduleAtFixedRate(() -> System.out.println("Bip"), 0, 1, TimeUnit.SECONDS);
Thread.sleep(5000);
pool.shutdown();
```

Affiche `Bip` toutes les secondes pendant 5 s, puis s'arrête (voir capture).

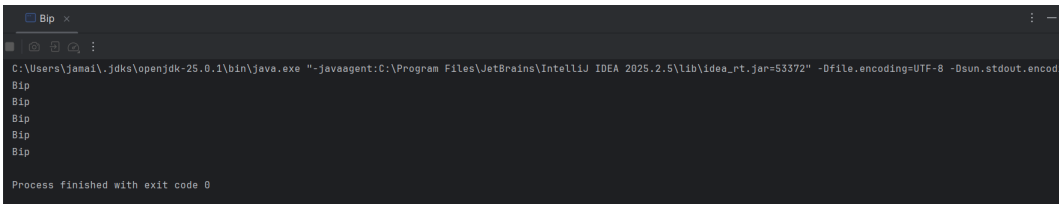


FIG. 11 : « Bip » répété toutes les secondes pendant 5 s

7 Le départ de la course

5 coureurs (5 threads) s'échauffent, attendent un signal commun, puis partent **en même temps**. L'arbitre (thread `main`) attend que tous soient prêts, donne le départ, puis attend l'arrivée de tous. Classes : `LigneDeDepart` (objet partagé), `Classement`, `Coureur` (thread), `Course`.

7.1 Étape 1 — `LigneDeDepart` (TODO 1 à 4)

```
public synchronized void seDeclarerPret(String nom) {
    nbPrets++;
    System.out.println(nom + " est prêt (" + nbPrets + "/" + nbCoureurs + ")");
    notifyAll();                                // TODO 1
}
public synchronized void attendreTousPrets() throws InterruptedException {
    while (nbPrets < nbCoureurs) { wait(); }    // TODO 2
    System.out.println("Arbitre : tout le monde est prêt !");
}
public synchronized void donnerDepart() {
    departDonne = true;
    System.out.println(">>> PAN ! Départ ! <<<");
    notifyAll();                                // TODO 3
}
public synchronized void attendreDepart() throws InterruptedException {
    while (!departDonne) { wait(); }            // TODO 4
}
```

7.2 Étapes 3 et 4 — `Coureur` et `Course`

Points clés : `super(nom)` + initialisation des attributs ; `seDeclarerPret` puis `attendreDepart` ; dans `Course`, boucle `for` pour créer/démarrer les 5 coureurs, puis `join()` sur chacun. La capture ci-dessous montre un déroulement complet. **Vérifications :** aucun `s'élance` n'apparaît avant `PAN !` ; les 5 `s'élance` apparaissent juste après ; les positions vont de 1 à 5 sans doublon ; `course terminée` s'affiche en dernier.

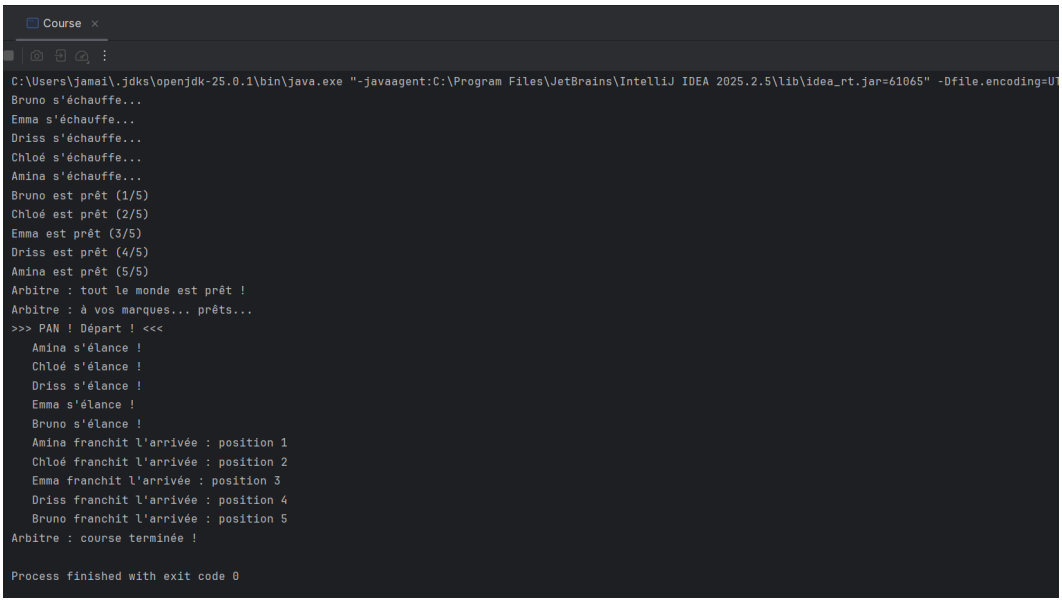


FIG. 12 : Déroulement complet de la course

7.3 Étape 2 — Question sur `Classement`

Sans `synchronized` dans `arriver()`, deux coureurs arrivant en même temps pourraient lire la même `prochainePosition` → **doublon** et position sautée.

7.4 Étape 5 — Expériences

1. **`notifyAll()` → `notify()` dans `donnerDepart()` :** `notify()` ne réveille qu'**un seul** coureur ; les autres restent bloqués, le `join()` ne se termine jamais → le programme ne se termine pas.
2. **Supprimer `attendreTousPrets()` dans `Course` :** l'arbitre peut donner le départ avant que tous soient prêts. Les coureurs encore en échauffement **ne sont pas bloqués pour toujours** : en arrivant, leur `attendreDepart()` voit `departDonne == true` (grâce au `while`) et ils partent aussitôt.
3. **`while (!departDonne)` → `while (true)` :** la condition de sortie disparaît ; chaque coureur reste **bloqué pour toujours** dans `wait()` → la course ne démarre jamais.

7.5 Étape 6 — Comprendre les choix

1. **Pourquoi `departDonne` est-il indispensable ?** C'est la **condition d'état** qui mémorise que le départ a eu lieu. Sans elle, un coureur qui fait `wait()` après un `notifyAll()` déjà passé attendrait un signal qui ne reviendra jamais (réveil manqué).
2. **Pourquoi `while` et non `if` ?** À cause des réveils intempestifs (*spurious wakeups*) et des réveils multiples : au réveil il faut **re-vérifier la condition**.
3. **Un coureur dans `wait()` consomme-t-il du CPU ? Non** : `wait()` endort le thread et libère le verrou. Une attente active `while (!departDonne) {}` occuperait un cœur à 100 % inutilement.
4. **Pourquoi `notifyAll()` plutôt que `notify()` ?** Plusieurs threads attendent sur le **même objet** ; `notify()` n'en réveillerait qu'un (peut-être pas le bon), `notifyAll()` les réveille tous et chacun re-teste sa condition.

7.6 La version moderne — `CountDownLatch` (bonus)

`LigneDeDepart` réécrite avec deux `CountDownLatch`, sans `synchronized`, `wait()` ni `notifyAll()` :

```java
private final CountDownLatch prets;            // new CountDownLatch(5)
private final CountDownLatch depart = new CountDownLatch(1);
// coureur prêt            : prets.countDown();
// arbitre attend          : prets.await();
// arbitre donne le départ : depart.countDown();
// coureur attend le départ: depart.await();
```

Les classes `Coureur`, `Classement` et `Course` ne changent pas. Même comportement, code beaucoup plus simple.

8 FAQ — Erreurs fréquentes rencontrées

Symptôme	Cause	Solution
`unreported exception InterruptedException`	`sleep/join/wait/get` lèvent une exception vérifiée	`throws InterruptedException` ou `try/catch`
`IllegalThreadStateException`	`start()` appelé deux fois	créer un nouvel objet `Thread`
`IllegalMonitorStateException`	`wait()/notifyAll()` hors d'un bloc `synchronized`	appeler dans une méthode `synchronized`
variable non `final` dans une lambda	usage de `i` (variable de boucle)	copier avant : `int numero = i;`
`ArrayIndexOutOfBoundsException`	c'est le bug à observer (3.1)	le noter dans le tableau
Durée ≈ 0 ms (ex. 1)	`join()` oublié	ajouter les `join()`
Le programme ne s'arrête jamais	pool non arrêté / `notifyAll()` oublié	`shutdown()` / `notifyAll()`

Conclusion

Ce TD a permis de manipuler les deux façons de créer des threads (`Runnable` et `extends Thread`), d'observer leur cycle de vie, et surtout de comprendre les **deux garanties distinctes** de la concurrence :

- la **visibilité** (`volatile`) — voir les écritures des autres threads ;
- l'**atomicité** (`synchronized`, `AtomicInteger`) — rendre indivisibles les opérations composées comme `c++` ou « vérifier puis ajouter ».

Les outils de haut niveau (`ExecutorService`, `Future`, `CountDownLatch`) simplifient grandement l'écriture de code concurrent correct.